SYSTEM GUIDE · LEVEL 1 · 2 NOVEMBER 2026

How Airspace Monitor Works

Architecture, data flows and the two build tracks

A guided tour from the collector on a home PC to the map, the arrival board and the guarded AI chat — plus a practical split of the work between Track A and Track B.

This is the companion to the system documents `00–11`; the documents remain the source of truth.

Solid boxes = running or committed. Dashed boxes = to build before the core is complete.

The system is deliberately small: one collector, one VM, one database, one scheduler.

The main path

Follow one snapshot from the sky to the browser. The top row is the collector and ingest path; the bottom row is what the platform does with the data.



The one-sentence version. A home PC turns OpenSky snapshots into durable batches; the VM turns those batches into a time-series database, flights, events, aggregates and predictions; the web app turns the latest state into a map, a board and an AI answer.

What is running today (2 Nov 2026)

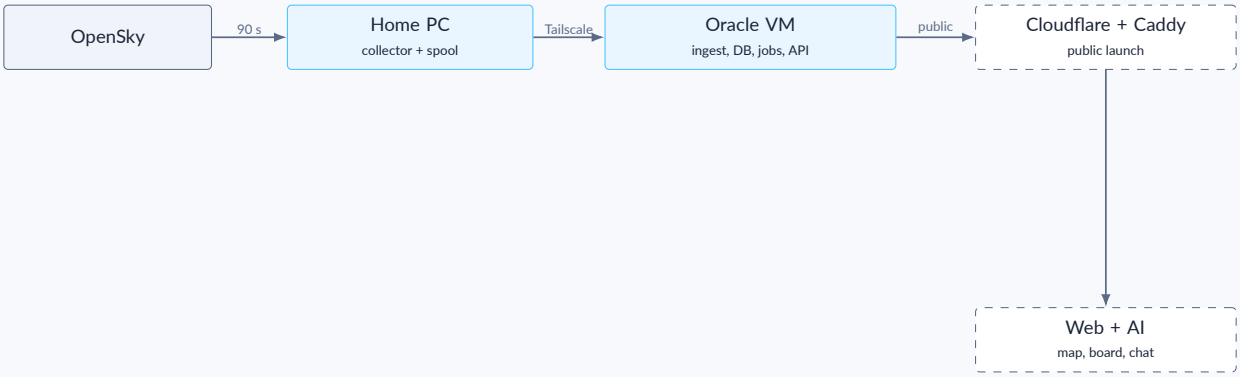
- pc-daniel collector, spool and uploader
- Oracle A1 VM, Tailscale, Docker
- TimescaleDB + PostGIS, migrations
- ingest-api and collector admin CLI
- 8.75M state vectors and 119k flight-list rows ingested

What the finished core adds

- Aggregates, Dagster jobs, DQ, Grafana
- public-api, SSE, web map and panels
- stream-worker events and ETA inference
- MLflow, ETA model, route lookup
- briefing, guarded chat, golden-set eval

Where everything runs

The system spans four places: a home connection, one Oracle VM, Cloudflare's edge before a public launch, and a few external services. The table is the quick reference; the two boxes below show the current private path and the path added for public launch.



Component	Runs on	Responsibility	Status
Collector	Home PC	Poll OpenSky on 90 s grid; spool every response; upload oldest-first; daily flight lists	Running
Spool + archive	Home PC / R2	Local durability, Parquet compaction, off-PC archive	Partial
ingest-api	Oracle VM	Authenticate, validate, normalise, H3, insert batches idempotently	Running
PostgreSQL	Oracle VM	TimescaleDB + PostGIS; raw, curated, aggregates, predictions, LLM data	Running
stream-worker	Oracle VM	Per-snapshot squawk/go-around events and ETA inference	To build
Dagster	Oracle VM	Windowed jobs: flights, labels, METAR, aggregates, anomaly, briefing, DQ, backup	To build
public-api	Oracle VM	Read API, SSE stream, chat endpoint	To build
MLflow	Oracle VM	Experiment tracking and model registry; private	To build
Grafana	Oracle VM	Ops, data-quality and model dashboards; externally shared	To build
Caddy + Cloudflare	Edge	TLS, DNS, proxy, rate limits; public hostnames	Later (private now)

Current path (private)

1. OpenSky → collector on pc-daniel
2. collector writes gzipped batches to spool
3. uploader posts over Tailscale to ingest-api
4. ingest writes raw rows and batch metadata
5. no inbound ports; SSH and admin over Tailscale

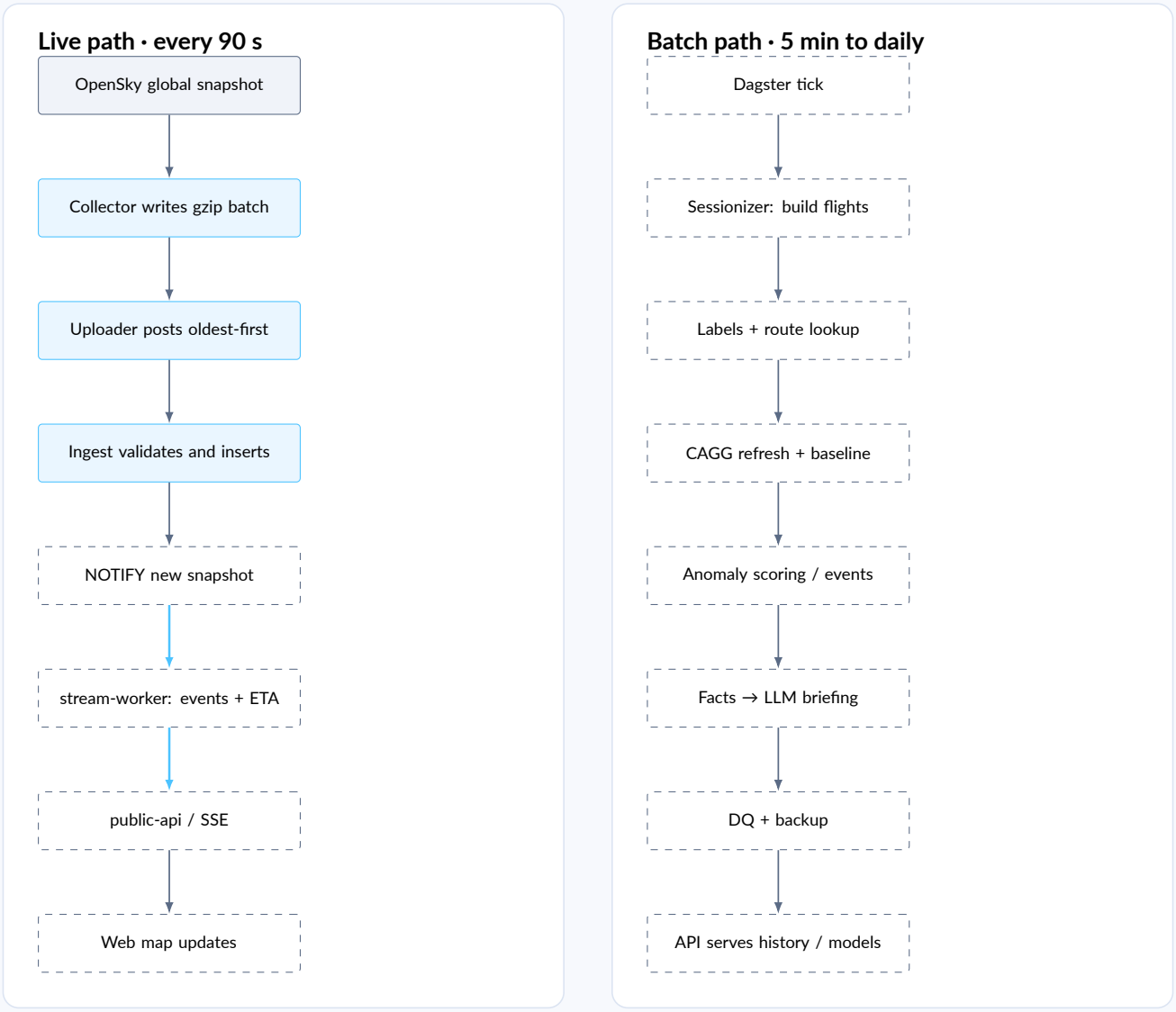
Added before public launch

1. Cloudflare DNS, proxy and rate limits
2. Caddy with Cloudflare Origin CA certificate
3. public-api, web app and Grafana hostnames
4. public web fetches the latest snapshot by `?t=`
5. OpenSky written licence confirmed first

The boundary that matters. OpenSky sees a residential IP, not Oracle. The VM never calls OpenSky. The collector is the only component that needs the OpenSky client secret and the only one that writes the spool. The server can be down without losing already-fetched data; it catches up later.

Two flows: live and batch

The system has one fast path for the map and one slower path for history, models and briefings. Keeping them separate is what makes the 90-second feed cheap.

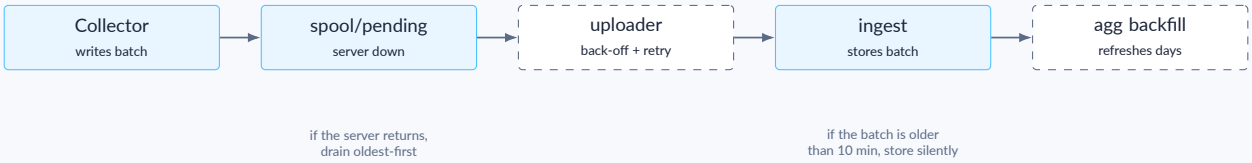


Catch-up path. If the server or network is down, the collector keeps writing to `spool/pending`. The uploader retries with back-off and drains oldest-first when the server returns. Batches older than the 10-minute live window are stored but do not notify, so the map and stream are never re-wound; `agg_backfill_refresh` materialises the affected days later. Manual `flights_rebuild` and `events_rebuild` cover the derived tables.

Why the split matters. The live path has a hard latency budget: a new snapshot should reach the map in seconds, and only the latest snapshot matters. The batch path can take minutes and can look backwards. Sharing one database but not one latency budget is what keeps the 90-second feed cheap and the history correct.

Failure and recovery

The system is designed to keep the last good state visible and to recover data later, rather than trying to pretend nothing happened.



Trigger	Immediate behaviour	Recovery
OpenSky 429 or timeout	Skip the slot, spool a failed envelope, obey `Retry-After`; the next slot is unaffected	Next slot; failed envelope explains the gap
Server unreachable	Pending spool grows; uploader backs off; map stays on the last good snapshot	Drain spool oldest-first; no live rewind
Batch older than 10 min	Stored silently; no SSE and no stream-worker	`agg_backfill_refresh`; manual re-score if needed
Stream worker down	Notifications are not durable; recent events can be missed	Manual `events_rebuild` over the window
Data-quality failure	`dq_fail` event and Grafana red status	Investigate; the UI does not hide the gap
Home PC off	No new snapshots; the last snapshot ages out of the live window	Resume when the PC returns; gap is visible

What to monitor

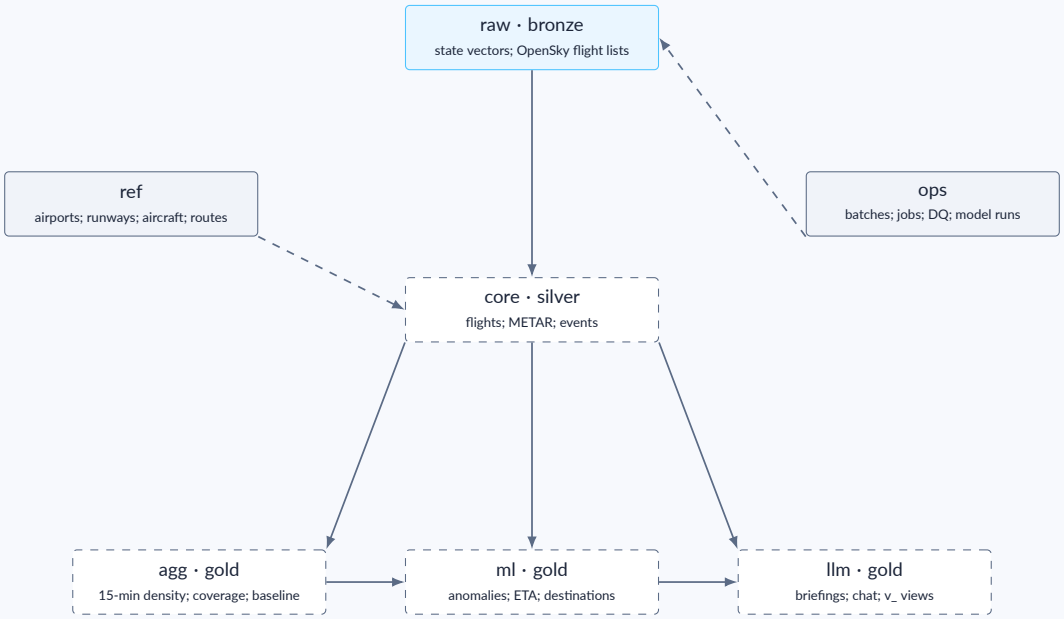
- `/health`: DB reachable; last snapshot age
- `/stats`: aircraft now, collectors online, rows today
- Grafana: slot coverage, gaps, ingest lag, credits
- DQ table: pass / warn / fail per check
- Spool: pending count, oldest pending age

What is intentionally not automatic

- Anomaly history is not re-scored when late data arrives.
- Stream events missed during a long outage need `events_rebuild`.
- Flights touched by late points need `flights_rebuild`.
- These are documented limits, not silent failures.

Inside the data platform

The database is organised as a medallion pipeline: raw data first, then curated entities, then aggregates and predictions. The chat is allowed to see only the final read-only views.

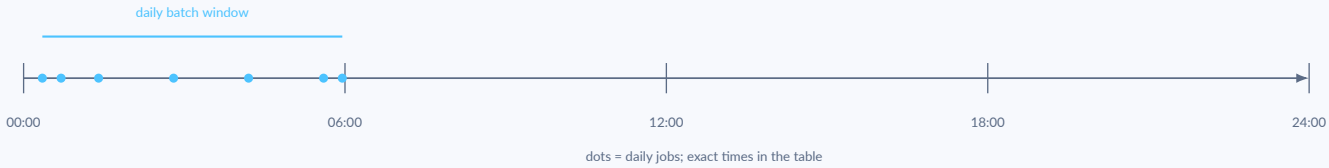


Schema	Main tables	Written by	Used for
`raw`	`state_vectors`, `opensky_flights`	ingest-api	source of truth; archives
`ref`	airports, runways, aircraft, airlines, countries, callsign routes	Dagster	joins; route lookup; wind
`core`	flights, METAR, events	stream-worker, Dagster	the product's entities
`agg`	`traffic_h3_15m`, coverage, baseline	Timescale, Dagster	density and anomalies
`ml`	anomalies, ETA, destination predictions	stream-worker, Dagster	map, board, card
`llm`	briefings, chat log, `v_*` views	API, Dagster	briefing and chat
`ops`	collectors, batches, jobs, DQ, model runs	all services	operations and audit

Keys are the contract. `batch_id` makes uploads idempotent. `dedupe_key` keeps events unique. `flight_id` is assigned once and never changes. Natural keys and explicit job windows let every scheduled job be re-run safely.

What runs when

The live path is continuous; the rest is a set of windows. The cadence is chosen around OpenSky credits, the 90 s source, and how quickly the product needs to react.



Cadence	Job / action	Why this cadence
90 s	Collector fetch + ingest + stream-worker + SSE	OpenSky credit budget: 960 global calls/day at 4 credits
5 min	`flights_sessionizer`, destination inference, gap check	flights and the card need to feel current
15 min (+2)	`anomaly_scoring` on the closed bucket	heatmap and anomaly events
30 min	`metar_fetch`, `llm_briefing`	weather and briefing are slower-moving
Hourly	`agg_backfill_refresh`, `dq_checks`	late data and health
Daily	baseline (02:00), backup (04:00), flight lists (06:00), labels (08:00), routes (08:30), compaction (00:30), R2 (01:00)	one pass over the previous day
Weekly	`train_eta` Sunday 03:00	enough new arrivals to retrain

The live promise

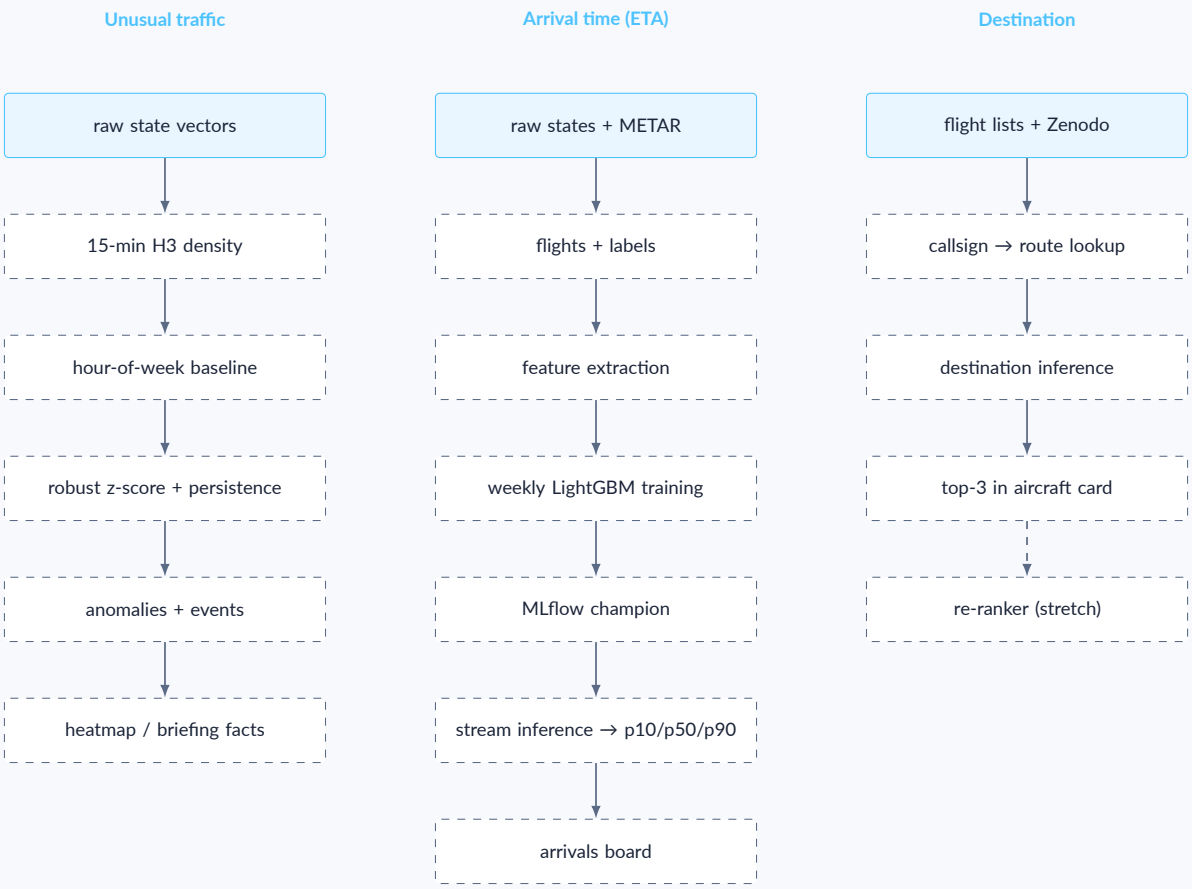
- At most one snapshot every 90 s.
- The map is never rewound to old data.
- The last good snapshot stays visible during an outage.
- Every gap is measurable.

The batch promise

- Every job has an explicit `[start, end)` window.
- Every job is idempotent for that window.
- Late data is caught up, not hidden.
- Training never runs in CI; only on the server.

How the ML works

Three different jobs: unusual traffic (unsupervised), arrival time (supervised quantile regression), and destination (a deterministic route lookup in the core, with a re-ranker as stretch).

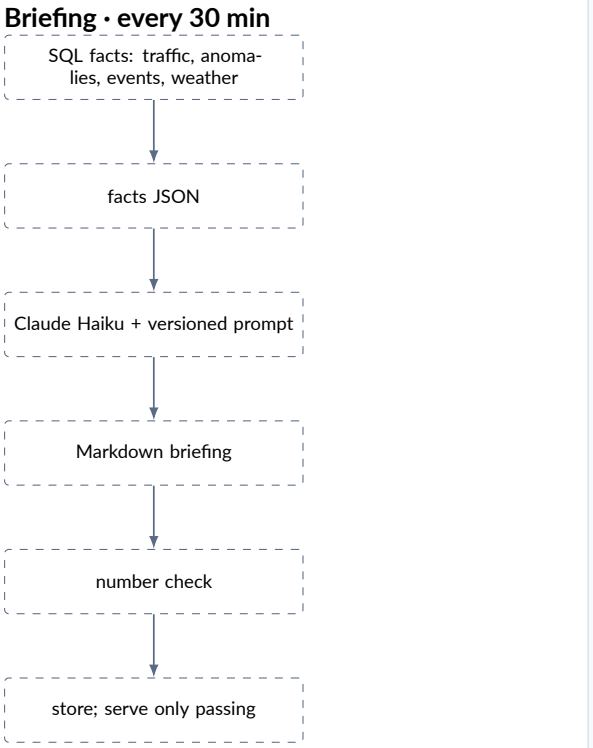


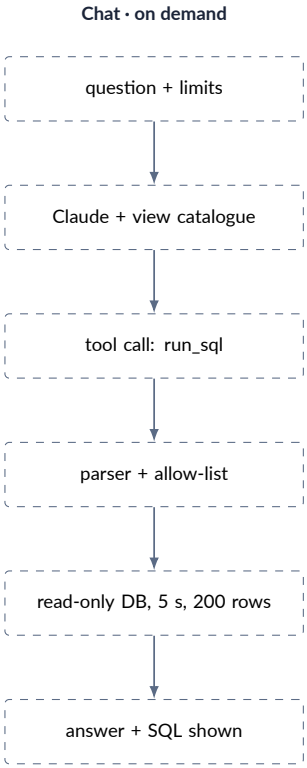
Model	Baseline	Evaluation	Success
Unusual traffic v1	median/MAD by hour of week	synthetic injection; flags/day; real-event list	useful precision/recall, not a fixed score
ETA	B0 distance/speed; B1 distance-band median	time-based hold-out; MAE; p10–p90 coverage	p50 MAE beats B1
Destination lookup	nearest airport ahead	top-1 / top-3 at 10/50/90% progress	beat nearest-airport baseline

Train and serve use the same feature code. Feature functions live in `libs/airspace_common` and `ml/`; training extracts Parquet with DuckDB, serving reads Postgres. MLflow stores the run, the metrics and the artifact. Only the `champion` alias is served; MLflow stays private.

How the LLM layer works

Two features, one rule: the model writes prose, but it does not invent facts and it does not get unrestricted SQL.





Guardrails

- `llm_readonly`: SELECT only on `llm.v_*`.
- One SELECT, allow-listed functions, no INTO/locking.
- 5 s statement timeout; 200-row cap; 3 tool calls.
- SQL always shown; rejected queries count as a call.
- 10 questions/hour/IP; 200/day; $40/month cap.

Why the number check exists The briefing may only use numbers from the facts JSON. Every number in the output is matched back to a fact (with rounding); a failed briefing is stored but never served. The previous passing briefing stays visible. This is the cheapest defence against a confident, wrong sentence.

The two build tracks

The split is by ownership, not by layer. Track A makes the data trustworthy; Track B makes it useful and visible. They meet at the data model and the API contract, and both review each other's PRs.

Track A · Data platform

- Collector, spool, uploader, flight lists
- ingest-api, auth, idempotency, H3
- DB, migrations, roles, retention, backups
- Dagster jobs, DQ, Grafana ops
- CI/CD, VM, Tailscale, R2, restore drill

Delivers: a database and a job graph that Track B can trust.

Track B · Product and ML

- public-api, SSE, web app, map layers
- ETA model, arrivals board, model cards
- Anomaly scoring, heatmap, events
- Destination lookup, aircraft card
- Briefing, chat, guardrails, golden-set eval

Delivers: the visible product and the measurable ML/LLM results.

Shared surface: contract first. `docs/03-data-model.md` defines the tables; `docs/05-api-contract.md` defines the endpoints. Change the contract first, then the code, and update the doc in the same PR. The unusual-traffic model is explicitly shared: Track A builds the aggregate and baseline, Track B builds scoring, the heatmap and the evaluation.

Handoff	Track A gives	Track B needs it for
Data model	`raw`, `core`, `agg`, `ml`, `llm` tables and keys	API queries, model labels, chat views
Aggregates	`agg.traffic_h3_15m`, coverage, baseline	anomaly scoring and heatmap
Flights	sessionizer, labels, route lookup	ETA training, arrivals board, card
API contract	endpoint shapes and units	web app, SSE, chat UI
Ops	health, DQ, backups, deploy	a product that can be trusted and shown

Current state (from 11-tasks) Track A is ahead: collector, VM, database, migrations, ingest-api and uploader are running; 8.75M state vectors are stored. Track B is mostly at the starting line: API, web, stream worker, Dagster, ML and LLM are still to build.

Open decision O1 Who owns Track A and Track B is still undecided. Decide it in week 0; the work split only works if each person has a clear primary track and both review the other's PRs.

Milestones and the critical path

The two tracks run in parallel, but they do not run independently: Track B's API and map need Track A's database, flights and aggregates. The critical path is collector → ingest → database → flights/labels → ETA → arrivals board.



Milestone	Track A	Track B
M0 · Foundations	collector v0, VM, DB	docs, mockup, contract review
M1 · Platform	ingest, migrations, Dagster, DQ, Grafana, CI, backup	API contract, web shell prep
M2 · Live map	stream worker, METAR, network	public API, map, events, deploy
M3 · Curated data	sessionizer, labels, routes, ref data	ETA data prep; anomaly design
M4 · ML	baseline, MLflow	anomaly, ETA, destination, model cards
M5 · LLM	support, views, deploy	briefing, chat, guardrails, eval
M6 · Polish	restore drill, docs	README, demo video, final report

Next five actions

1. Decide O1: who owns A and B.
2. Finish M0.13/M0.14: 48 h check, measured rows/day.
3. Land M1.10–M1.13: aggregates, compression, Dagster, Grafana.
4. Land M1.14–M1.16: CI, deploy, backup + restore drill.
5. Start Track B's API and map against the stable contract.

What not to do

- Do not start stretch before M5 is green.
- Do not open a public URL before the OpenSky licence is confirmed.
- Do not add a fourth ML model.
- Do not let M6 become feature time.